

CryptoScript DSL

Language Specification

Keywords • Grammar • Types • Operators

Team Members	
Avinash	24CS10009
Jinansh Dalal	24CS10016
Abhinav Tiwari	24CS10077
Nithish Kumar	24CS10106

1. Motivation

Python is currently the most used language for anything related to Cryptography. However, its general-purpose nature presents several shortcomings when expressing strict Cryptographic logic. We have identified several key shortcomings:

- **Data Conversion:** Cryptography requires conversions between strings, hex, integers, and bytes. In native Python, this process is unnecessarily clunky. While experienced developers learn to work around this, this poses a steep learning curve.
- **Finite Field Arithmetic:** Quite a few Cryptographic algorithms, notably RSA, operate almost entirely over finite fields. Implementing these in Python requires the repetitive application of the modulo operator (e.g., $(a * b) \% n$) or continually passing the modulus as an argument to functions like `pow()`. Failing to do so leads to logic errors and incorrect algorithm outputs.
- **Logic Operations:** Implementing symmetric algorithms that rely on bitwise rotations, substitutions (S-boxes), and permutations requires writing heavy code. Python lacks the specialized syntax to write these trivial operations elegantly.
- **Poor Readability of Nested Operations:** Cryptographic algorithms are data pipelines consisting of sequential transformations. Standard function calls quickly lead to nested, difficult-to-read code.

We will be building this DSL with these shortcomings in mind. By compiling this DSL down to Python, we retain full access to Python's arbitrary-precision integers and its large cryptographic libraries.

2. Features

1. Core Syntax

- **Python Foundation:** For loops, conditionals and basic mathematical operations, Python's functionalities will be used. We do not see the benefit of reinventing the wheel here.
- **Library Support:** Because there is a lot of support already existing in Python through the use of libraries, we have made a decision to support these Python libraries.

2. New Cryptographic Data Types

- **Native Hex and Bytes Types:** We introduce dedicated types for Hexadecimal and Byte data. This will improve readability, ensure typing, and make conversions between strings, hex, integers, and data easier.

3. Mathematical Operations

- **Modular Arithmetic Types:** Developers can define integers bound to a specific modulus. Standard arithmetic operators (+, -, ×) are overloaded to automatically apply the modulo operation.
- **Matrices:** Matrix types are introduced with built-in support for complex matrix arithmetic over finite fields.

4. Easier Data Flow

- **The Pipelining Operator (|>):** To address the unreadability of nested function calls, we introduce a pipeline operator. This will allow a developer to chain multiple steps cleanly, taking the output of the previous step and passing it as the argument to the next.

5. Declarative Cipher Construction

- **The Cipher Function Type:** We introduce a way to define a specialized set of functions specifically for designing ciphers. This allows users to define the behavior of a cipher using a straightforward set of rules (e.g., substitution, permutation, XOR mixing).
- **Automated Method Generation:** Since we enforce the definition of rules like so, we can generate the inverse of these transformations quite easily. Hence, defining a Cipher automatically generates both the `.encrypt()` and `.decrypt()` methods, saving the developer from writing redundant logic.

3. Keywords, Operators & Types

3.1 Keywords from Python

Inherited directly from Python with no changes to syntax or semantics.

Category	Keywords / Values
Logic / Boolean	<code>False</code> , <code>True</code> , <code>None</code> , <code>and</code> , <code>or</code> , <code>not</code>
Control Flow	<code>if</code> , <code>elif</code> , <code>else</code> , <code>for</code> , <code>while</code> , <code>break</code> , <code>continue</code> , <code>return</code>
Structure	<code>def</code> , <code>import</code> , <code>from</code> , <code>as</code> , <code>with</code> , <code>pass</code>
Pattern Matching	<code>match</code> , <code>case</code> , <code>in</code> , <code>is</code>

3.2 Custom Keywords

New keywords specific to CryptoScript — not present in standard Python.

Category	Keywords / Values
<code>cipher</code>	Defines a new cryptographic algorithm block
<code>encrypt</code>	Defines the forward-transformation block inside a cipher
<code>sbox</code>	Declares a substitution mapping (lookup table)
<code>pbox</code>	Declares a permutation mapping (position rearrangement)
<code>repeat</code>	Repeats a block of pipeline steps N times (for cipher rounds)

3.3 Operators from Python

Category	Keywords / Values
Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>//</code> <code>%</code> <code>**</code>
Assignment	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>//=</code> <code>%=</code> <code>**=</code> <code>&=</code> <code> =</code> <code>^=</code> <code>>>=</code> <code><<=</code>
Comparison	<code>==</code> <code>!=</code> <code><</code> <code>></code> <code><=</code> <code>>=</code>
Bitwise	<code>&</code> <code> </code> <code>^</code> <code>~</code> <code><<</code> <code>>></code>

Category	Keywords / Values
Delimiters	() [] { } , : . ; ->

3.4 Custom Operator — Pipeline |>

Chains transformations left to right. The output of each step becomes the input of the next, eliminating deeply nested function calls.

Category	Keywords / Values
> (Pipeline)	Chains transformations left to right. Output of previous step becomes input to next.

Without pipeline operator:

```
result = to_hex(xor(substitute(permute(data, my_pbox), my_sbox), key))
```

With pipeline operator:

```
result = data |> permute(my_pbox) |> substitute(my_sbox) |> xor(key) |> to_hex
```

3.5 Data Types

Category	Keywords / Values
Int	Arbitrary-precision integer
Bool	True or False
String	UTF-8 / ASCII text
Bytes	Raw byte array
Hex	Hexadecimal representation
ModInt[n]	Integer bound to modulus n. Operators +, -, ×, ** auto-apply mod n
Matrix[Int, r, c]	Matrix of integers with dimensions r × c
Matrix[ModInt[n], r, c]	Matrix over finite field mod n with dimensions r × c

3.6 Core Transform Primitives

Built-in callables for cryptographic data manipulation. Used inside cipher blocks and regular functions.

Category	Keywords / Values
substitute(IDENT)	Replace each byte using a substitution box (sbox)
permute(IDENT)	Rearrange bytes / bits using a permutation box (pbox)
xor(expr)	XOR two Bytes or Hex values together
rotl(expr, expr)	Rotate bits of value left by N positions
rotr(expr, expr)	Rotate bits of value right by N positions
pad(scheme)	Pad data to a multiple of block size – PKCS7 or ZERO
unpad(scheme)	Remove padding from data

3.7 Safe Type Conversions

Category	Keywords / Values
to_bytes(expr)	Convert Int / Hex / String → Bytes
to_le_bytes(expr)	Convert to Bytes in little-endian order
to_be_bytes(expr)	Convert to Bytes in big-endian order
to_hex(expr)	Convert Int / Bytes / String → Hex
to_string(expr)	Convert Bytes / Hex → readable String
to_int(expr)	Convert Bytes / Hex / String → Int

4. Grammar

The grammar is defined in Extended Backus-Naur Form (EBNF). Curly braces { } denote zero or more repetitions. Square brackets [] denote optional elements.

4.1 Program Structure

Program Structure	
program	::= { statement }
statement	::= import_stmt func_def cipher_def sbbox_def pbox_def
	var_decl assign_stmt if_stmt for_stmt while_stmt
	repeat_stmt match_stmt return_stmt with_stmt
	expr_stmt 'pass' 'break' 'continue'
block	::= NEWLINE INDENT { statement } DEDENT

4.2 Imports

Importing Libraries	
import_stmt	::= 'import' module_path
	'from' module_path 'import' name_list
module_path	::= IDENT { '.' IDENT }
name_list	::= IDENT { ',' IDENT }

4.3 Type Annotations

Types	
type	::= 'Int' 'Bool' 'String' 'Bytes' 'Hex'
	'ModInt' '[' INT_LIT ']'
	'Matrix' '[' matrix_elem_type ',' INT_LIT ',' INT_LIT ']'
matrix_elem_type	::= 'Int' 'ModInt' '[' INT_LIT ']'

4.4 Variable Declaration

Variable Declaration	
var_decl	::= IDENT ':' type ['=' expr] NEWLINE

4.5 SBOX and PBOX Declaration

SBOX and PBOX	
sbox_def	::= 'sbox' IDENT '=' '[' box_entry { ',' box_entry } ']' NEWLINE
pbox_def	::= 'pbox' IDENT '=' '[' box_entry { ',' box_entry } ']' NEWLINE
box_entry	::= INT_LIT HEX_LIT BYTES_LIT STRING_LIT BOOL_LIT

4.6 Function Definition

Function Definition	
func_def	::= 'def' IDENT '(' [param_list] ')' ['->' type] ':' block
param_list	::= param { ',' param }
param	::= IDENT ':' type ['=' expr]

4.7 Cipher Definition

Cipher Definition	
cipher_def	::= 'cipher' IDENT '(' [param_list] ')' ':' NEWLINE INDENT cipher_block DEDENT
cipher_block	::= 'encrypt' ':' NEWLINE INDENT pipeline_body DEDENT
pipeline_body	::= pipeline_expr NEWLINE
pipeline_expr	::= or_expr { ' >' pipeline_step }
pipeline_step	::= transform_call repeat_block func_call

Usage example:

```
sbox a1 = [ 0x63, 0x7c, 0x77, 0x7b, ... ]

cipher my_cipher(key: Bytes) -> Bytes:
  encrypt:
    data |> pad(PKCS7)
        |> repeat(4):
            xor(key)
        |> substitute(a1)
```

4.8 Repeat

Repeat	
<code>repeat_stmt</code>	<code>::= 'repeat' '(' expr ')' ':' block</code>
<code>repeat_block</code>	<code>::= 'repeat' '(' expr ')' ':' NEWLINE INDENT pipeline_expr NEWLINE DEDENT</code>

4.9 Custom Transforms

Transform Calls	
<code>transform_call</code>	<code>::= 'substitute' '(' IDENT ')'</code>
	<code> 'permute' '(' IDENT ')'</code>
	<code> 'xor' '(' expr ')'</code>
	<code> 'rotr' '(' expr ',' expr ')'</code>
	<code> 'rotl' '(' expr ',' expr ')'</code>
	<code> 'rotr' '(' expr ',' expr ')'</code>
	<code> 'pad' '(' padding_scheme ')'</code>
	<code> 'unpad' '(' padding_scheme ')'</code>
	<code> conversion_call</code>
<code>padding_scheme</code>	<code>::= 'PKCS7' 'ZERO'</code>
<code>conversion_call</code>	<code>::= ('to_bytes' 'to_le_bytes' 'to_be_bytes'</code>
	<code> 'to_hex' 'to_string' 'to_int') '(' expr ')'</code>

4.10 Control Flow

Conditionals and Loops	
<code>if_stmt</code>	<code>::= 'if' expr ':' block { 'elif' expr ':' block } ['else'</code> <code>':' block]</code>
<code>for_stmt</code>	<code>::= 'for' IDENT 'in' expr ':' block</code>
<code>while_stmt</code>	<code>::= 'while' expr ':' block</code>
<code>match_stmt</code>	<code>::= 'match' expr ':' NEWLINE INDENT { case_clause } DEDENT</code>
<code>case_clause</code>	<code>::= 'case' pattern ':' block</code>
<code>pattern</code>	<code>::= literal IDENT '_'</code>
<code>with_stmt</code>	<code>::= 'with' expr ['as' IDENT] ':' block</code>
<code>return_stmt</code>	<code>::= 'return' [expr] NEWLINE</code>

4.11 Assignment

Assignment	
assign_stmt	::= target assign_op expr NEWLINE
target	::= IDENT IDENT '[' expr ']' IDENT '.' IDENT
assign_op	::= '=' '+=' '-=' '*=' '/=' '//=' '%=' '**=' '&=' ' =' '^=' '>>=' '<<='
expr_stmt	::= expr NEWLINE

4.12 Expressions

Expression Hierarchy	
expr	::= pipeline_expr
pipeline_expr	::= or_expr { ' '>' pipeline_step }
or_expr	::= and_expr { 'or' and_expr }
and_expr	::= not_expr { 'and' not_expr }
not_expr	::= 'not' not_expr comparison
comparison	::= bitwise_or { comp_op bitwise_or }
comp_op	::= '==' '!=' '<' '>' '<=' '>=' 'in' 'is'
bitwise_or	::= bitwise_xor { ' ' bitwise_xor }
bitwise_xor	::= bitwise_and { '^' bitwise_and }
bitwise_and	::= shift { '&' shift }
shift	::= arith { ('<<' '>>') arith }
arith	::= term { ('+' '-') term }
term	::= factor { ('*' '/' '//' '%') factor }
factor	::= unary '**' factor unary
unary	::= ('~' '-' '+') unary primary
primary	::= atom { trailer }
trailer	::= '.' IDENT '[' expr ']' '(' [arg_list] ')'
func_call	::= IDENT '(' [arg_list] ')'
arg_list	::= arg { ',' arg }
arg	::= expr IDENT '=' expr

4.13 Atoms

Atoms	
atom	::= IDENT INT_LIT HEX_LIT BYTES_LIT STRING_LIT BOOL_LIT
	'None' '(' expr ')' list_lit
list_lit	::= '[' [expr { ',' expr }] '']
literal	::= INT_LIT HEX_LIT BYTES_LIT STRING_LIT BOOL_LIT 'None'

4.14 Literals

Literals	
INT_LIT	::= '0' [1-9] { DIGIT }
HEX_LIT	::= '0x' HEX_DIGIT { HEX_DIGIT }
BYTES_LIT	::= 'b' ''' { BYTE_CHAR } '''
STRING_LIT	::= '"' { CHAR } '"' "'" { CHAR } "'"
BOOL_LIT	::= 'True' 'False'

4.15 Terminals

Terminals	
IDENT	::= LETTER { LETTER DIGIT '_' }
LETTER	::= 'a'..'z' 'A'..'Z' '_'
DIGIT	::= '0'..'9'
HEX_DIGIT	::= DIGIT 'a'..'f' 'A'..'F'
CHAR	::= any Unicode character except '"' and '\' '\' Escape_seq
Escape_seq	::= '"' '\' 'n' 't' 'r'
BYTE_CHAR	::= '\x' HEX_DIGIT HEX_DIGIT any ASCII except '"' and '\'' '\\' Escape_seq
